TP 9 · INITIATION AU DÉVELOPPEMENT D'APPLICATION

Les Tests

dans le cycle de vie d'une application

Pourquoi un build réussi ne suffit pas : types de tests, outils Angular et Spring Boot, pipeline CI/CD et cas concret issu d'une user story.

5 minutes | Angular · Spring Boot | Code réel





AKALETE Koffi Levis

02 novembre 2026

POURQUOI TESTER ?

« Ça compile » ne veut pas dire « ça marche »

Le build prouve la faisabilité ; le test prouve le comportement.

BUILD

SUCCESS

Ce qu'il garantit

- **Compilation** : le code compile.
- **Artefact** : dist/ ou .jar est généré.

Faisabilité technique — pas la qualité.

puis

TESTS

GO / NO-GO

Ce qu'il vérifie

- **Comportement** : la règle métier est respectée.
- **Régression** : l'existant ne casse pas.

Qualité fonctionnelle — avant la livraison.

DEFINITION OF DONE

Code review ✓ + Build ✓ + Tests ✓ = prêt pour la Release

Sans tests, une régression peut atteindre la production sans être détectée.

DÉFINITIONS

Le test et le rapport de test

Le test compare l'attendu au réel ; le rapport transforme cette exécution en preuve.

LE TEST

Comparer l'attendu au réel

Un contrôle qualité sur un périmètre donné : on prépare un cas, on exécute, on vérifie le résultat.

`Arrange → Act → Assert`

LE RAPPORT

La preuve de l'exécution

Liste les tests lancés, leurs statuts, les erreurs, la durée et la couverture de code.

`GO / NO-GO → décision`

VERDICT

Tous verts → Release autorisée. **Un échec** → pipeline bloqué.

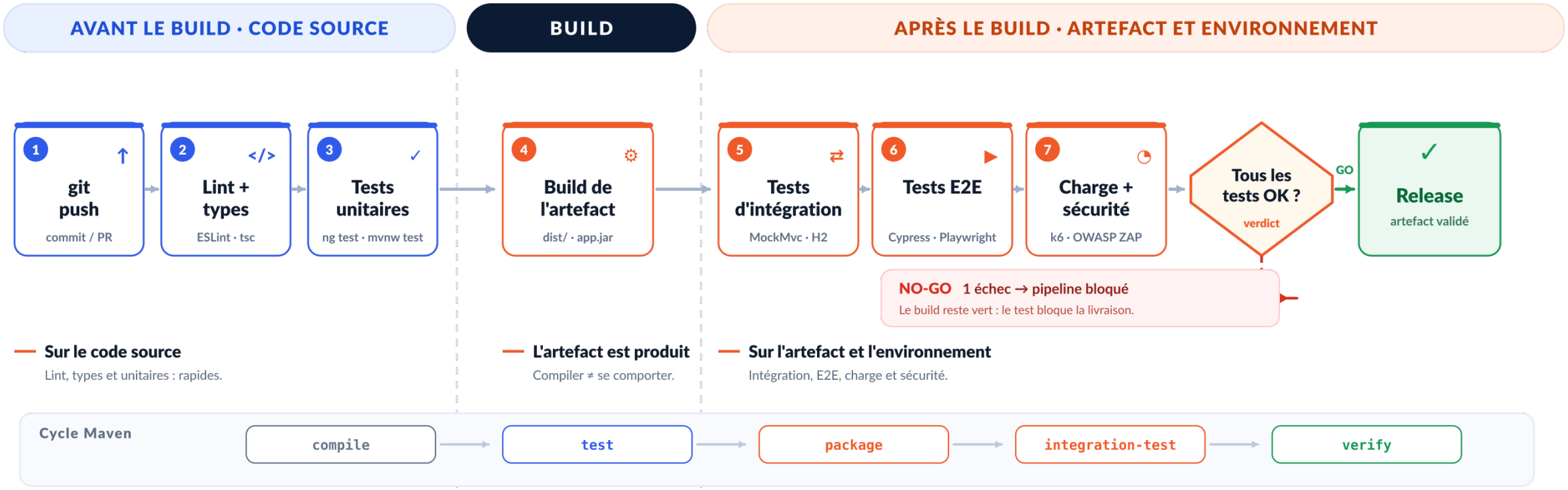


Extrait de rapport · JUnit XML · HTML · JSON · couverture JaCoCo

PIPELINE CI/CD

Où placer les tests dans le pipeline ?

Les tests unitaires précèdent le build ; intégration, E2E, charge et sécurité valident ensuite l'artefact et son environnement.



GO — tous les tests passent

La Release est autorisée.

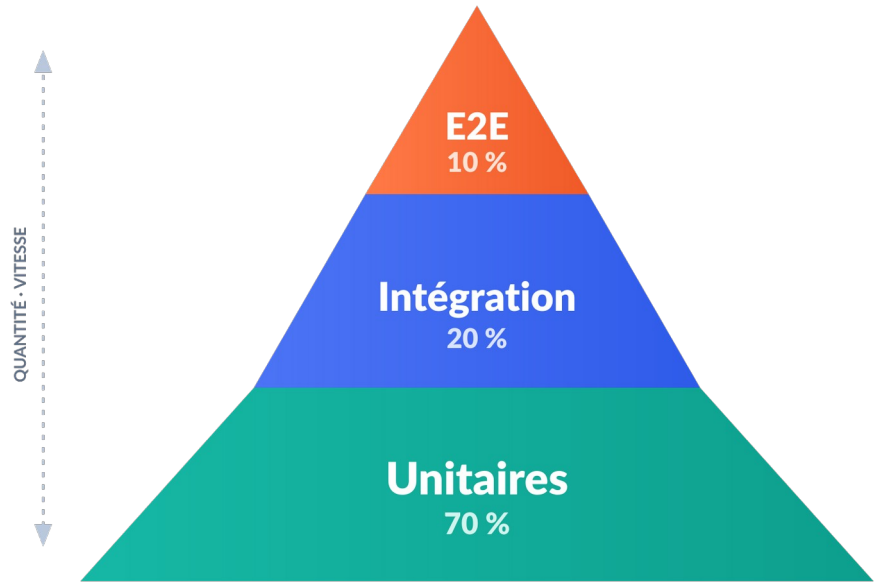
NO-GO — un seul test échoue

Le pipeline est bloqué : retour au code.

STRATÉGIE

La pyramide des tests

Beaucoup de tests rapides à la base, peu de tests lents au sommet.



UNITAIRE ≈ 70 % · rapides, isolés

Jasmine / Karma · JUnit 5 + Mockito

```
ng test  ·  ./mvnw test
```

INTÉGRATION ≈ 20 % · composants assemblés

TestBed · MockMvc · Testcontainers

```
ng test  ·  ./mvnw verify
```

E2E ≈ 10 % · parcours critiques

Cypress · Playwright

```
npx cypress run
```

RÉGRESSION

Rejouer les tests existants à chaque changement.

C'est le filet de sécurité du pipeline : une fonctionnalité qui marchait ne doit pas casser.

Unitaire et intégration : isoler, puis assembler

Deux niveaux, deux commandes : ng test et ./mvnw verify.

UNITAIRE `ng test` unité isolée

INTÉGRATION `./mvnw verify` couches réelles

auth.service.spec.ts — ng test

```ts
// auth.service.spec.ts
describe('AuthService', () => {
  let service: AuthService;
  let http: HttpTestingController;
  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient(),
        provideHttpClientTesting()],
    });
    service = TestBed.inject(AuthService);
    http = TestBed.inject(HttpTestingController);
  });
  it('stocke le jeton', () => {
    service.login({ email: 'ada@exemple.fr',
      password: 'secret' }).subscribe((r) => {
      expect(r.token).toBe('jwt-123');
    });
    http.expectOne('/api/auth/login')
      .flush({ token: 'jwt-123' });
  });
});
```

AuthControllerIT.java — ./mvnw verify

```java
// AuthControllerIT.java
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class AuthControllerIT {
  @Autowired MockMvc mvc;
  @Autowired UserRepository users;
  @Autowired PasswordEncoder encoder;
  @BeforeEach
  void setUp() {
    users.save(new User("ada@exemple.fr",
      encoder.encode("secret")));
  }
  @Test
  void login_valide_retourne200() throws Exception {
    mvc.perform(post("/api/auth/login")
        .contentType(APPLICATION_JSON)
        .content("""
          {"email":"ada@exemple.fr","password":"secret"}"""))
      .andExpect(status().isOk())
      .andExpect(jsonPath("$.token").isNotEmpty());
  }
}
```

Unitaire = la logique seule. Intégration = contrôleur + service + base.

End-to-end et régression

L'E2E simule un utilisateur réel ; la régression rejoue l'existant.

login.cy.ts — npx cypress run

```ts
// cypress/e2e/login.cy.ts
describe('Connexion', () => {
  it('redirige vers le dashboard', () => {
    cy.intercept('POST', '/api/auth/login', {
      statusCode: 200,
      body: { token: 'jwt-123' },
    }).as('login');
    cy.visit('/login');
    cy.get('[data-cy=email]')
      .type('ada@exemple.fr');
    cy.get('[data-cy=password]').type('secret');
    cy.get('[data-cy=submit]').click();
    cy.wait('@login');
    cy.url().should('include', '/dashboard');
  });
});
```

login.feature — Gherkin

```gherkin
# features/login.feature
Fonctionnalité : Connexion

  Scénario : Identifiants valides
    Étant donné un utilisateur inscrit
    Quand il soumet le formulaire
    Alors il est redirigé vers "/dashboard"
    Et un jeton JWT est stocké
```

SCÉNARIO	1 comportement observable
IMPLÉMENTATION	MockMvc + Cypress
PREUVE	Surefire XML + HTML

RÉGRESSION

Rejouer toute la suite à chaque changement.

C'est le filet de sécurité : une fonctionnalité qui marchait ne doit pas casser.

TESTS NON-FONCTIONNELS

Performance, charge et sécurité

Ces tests mesurent le « comment », pas seulement le « quoi ».

login-load-test.js — k6 run

```
// k6/login-load-test.js
import http from 'k6/http';
import { check } from 'k6';
export const options = {
  vus: 50, duration: '1m',
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.01'],
  },
};
export default function () {
  const res = http.post(`${__ENV.API}/auth/login`,
    JSON.stringify({ email: 'ada@exemple.fr',
      password: 'secret' }),
    { headers: { 'Content-Type': 'application/json' } });
  check(res, { 'HTTP 200': (r) => r.status === 200 });
}
```

50
utilisateurs virtuels

p95 < 500 ms
temps de réponse

< 1 %
taux d'erreur

Seuils explicites : la CI transforme une mesure en décision GO / NO-GO.

PERFORMANCE / CHARGE

Temps de réponse, débit, résistance sous charge.

JMeter · Gatling · k6

`Rapport : dashboard HTML`

SÉCURITÉ

Dépendances vulnérables, en-têtes, OWASP Top 10.

```
npm audit --audit-level=high
zap-baseline.py -t https://…
```

`Rapport : HTML`

CAS CONCRET

De la user story au rapport d'échec

La story décrit le besoin ; le Gherkin le rend exécutable ; le rapport juge.

US-12 · SE CONNECTER

« En tant qu'utilisateur non authentifié, je veux me connecter afin d'accéder à mon tableau de bord. »

Compte existant | Redirection /dashboard

login.feature — Gherkin

```gherkin
# features/login.feature
Scénario : Identifiants valides
  Étant donné un utilisateur inscrit
  Quand il soumet le formulaire
  Alors il est redirigé vers "/dashboard"
  Et un jeton JWT est stocké
```

IMPLÉMENTATION

MockMvc (intégration) + Cypress (E2E)

```
./mvnw verify   ·   npx cypress run
```



Le test attend 401, l'application renvoie 200 : le rapport localise le bug.

SYNTHÈSE

Un build réussi ne suffit pas

Quatre réflexes, une seule décision : GO ou NO-GO.

01 Tester, c'est prouver

Comparer l'attendu au réel.

02 La pyramide guide l'effort

70 % unitaires, 20 % intégration, 10 % E2E.

03 Le rapport donne le verdict

GO autorise la Release ; un échec bloque.

04 Fonctionnel + non-fonctionnel

Le « quoi » et le « comment ».

Code review ✓ + Build ✓ + Tests ✓ = prêt pour la Release

Livrer en confiance, pas seulement livrer.

RÉFÉRENCES

Angular · `angular.dev/guide/testing`

Spring · `docs.spring.io/spring-boot/reference/testing`

Cypress · `docs.cypress.io`

k6 · `k6.io/docs`

JUnit 5 · `junit.org/junit5`

OWASP ZAP · `zaproxy.org`